



Vehicle platform

Of ECUs, microcontrollers and simulators...



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

High Performance
Real Time **Lab**



Course outline

Lesson #1

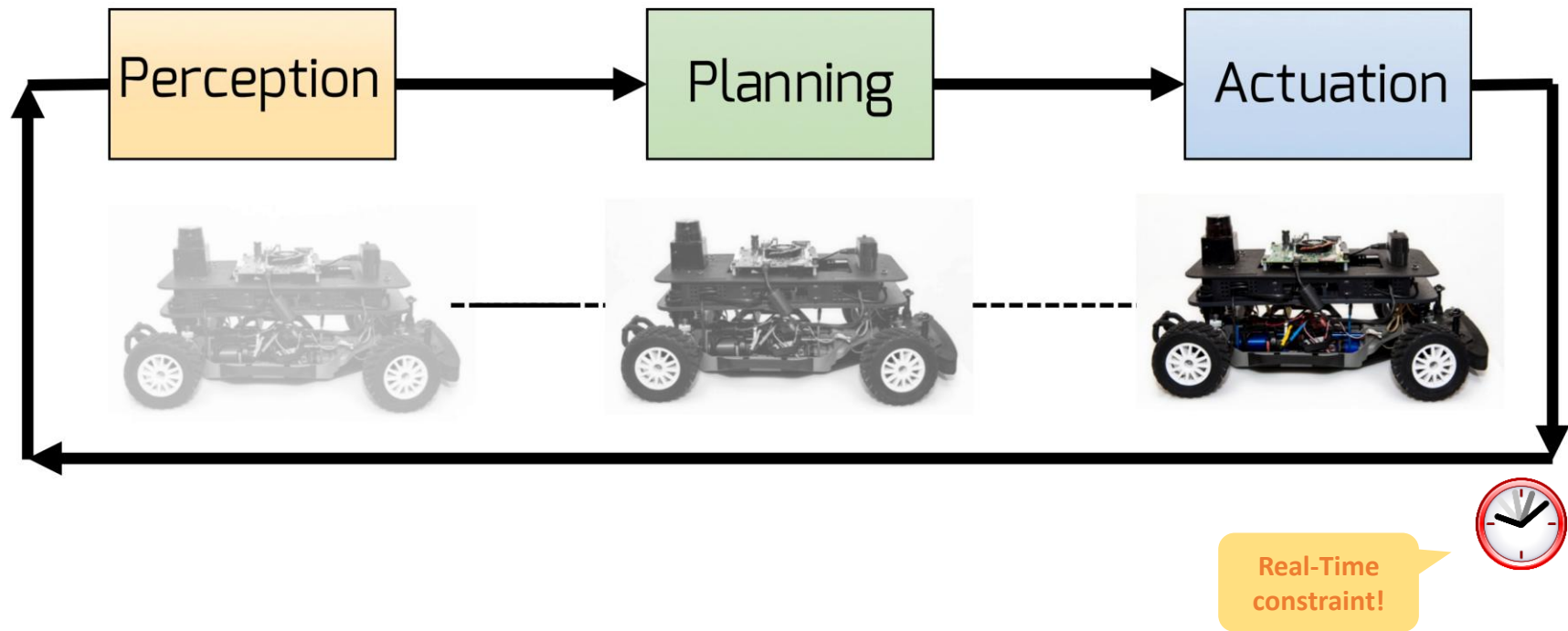
- › Intro course + basics of AD
- › Perception and AI
- › Hardware platforms

Lesson #2

- › Cars and the city
- › Navigation: planning and control
- › Let's build the car

Assignments

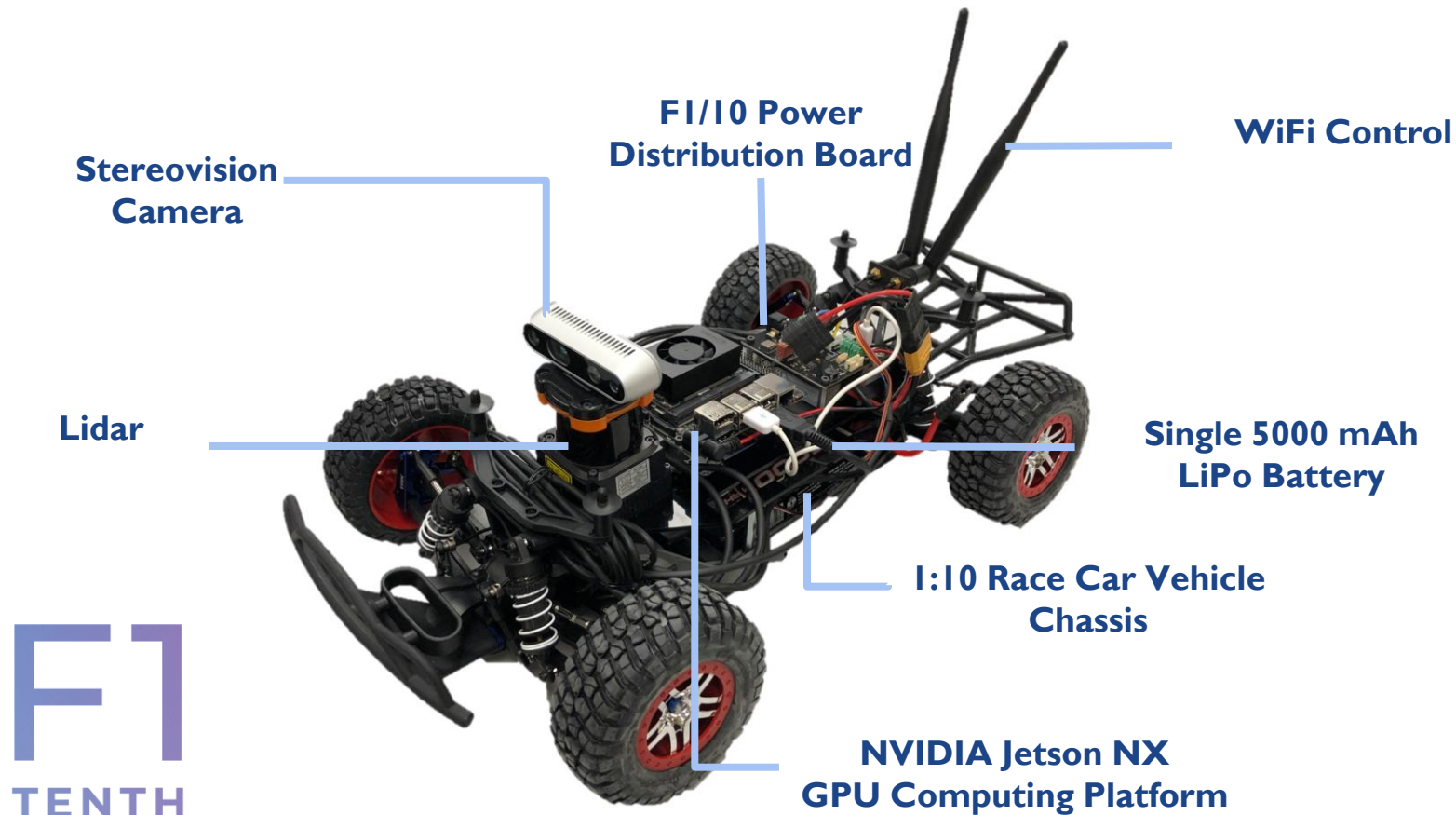




(At least) three stages – F1/10



The F1/10 platform



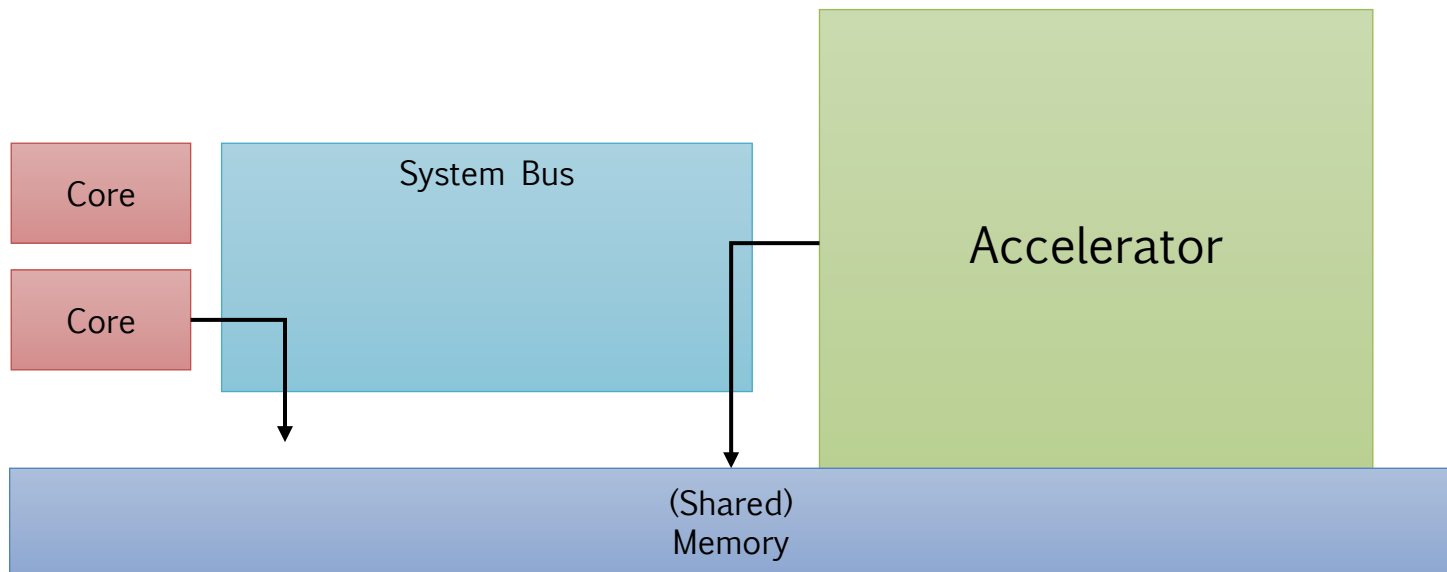


On-board computer - template

Aka: Domain controller, aka: ECU, aka: Zonal controller, aka ...

Embedded heterogeneous platform

- › Multi-core + data cruncher accelerator
- › Shall employ safety core (Aurix?) to enable automotive ISO26262/ASIL-D compliancy





NVIDIA Jetson Orin

6/8 core ARM host

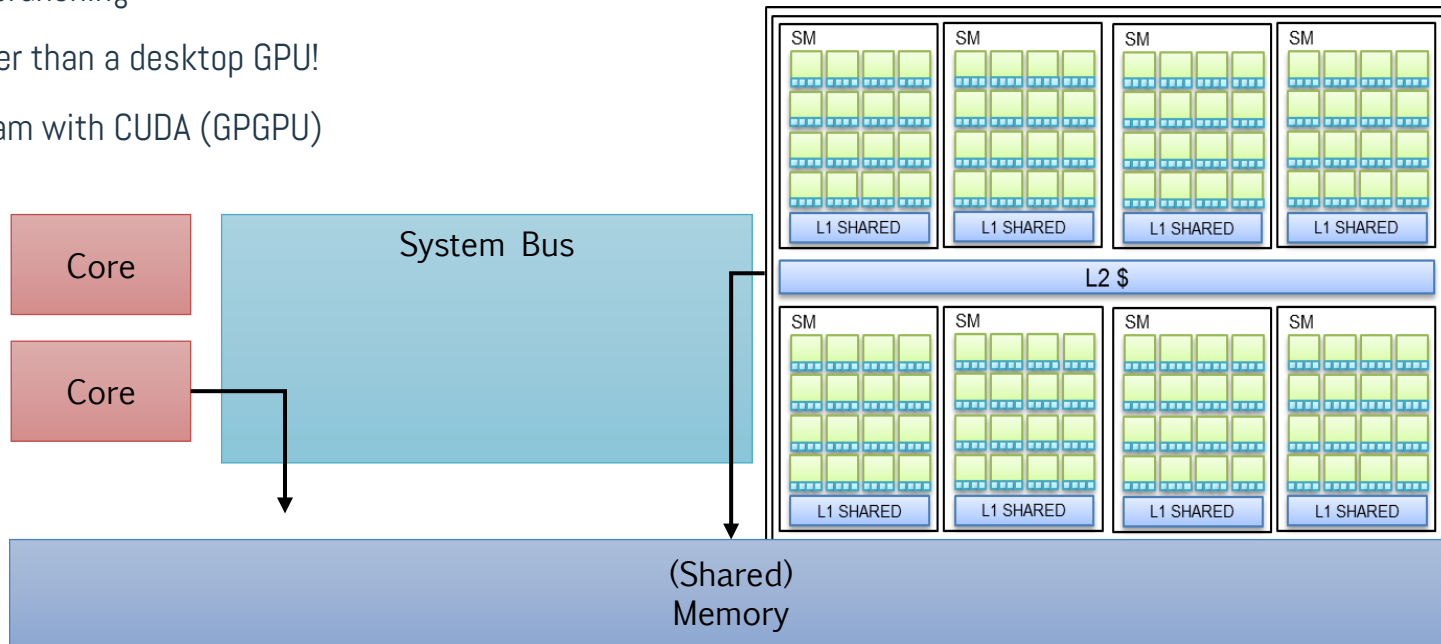
› For control-based workload

Embedded GPU

› Data crunching

› Smaller than a desktop GPU!

› Program with CUDA (GPGPU)





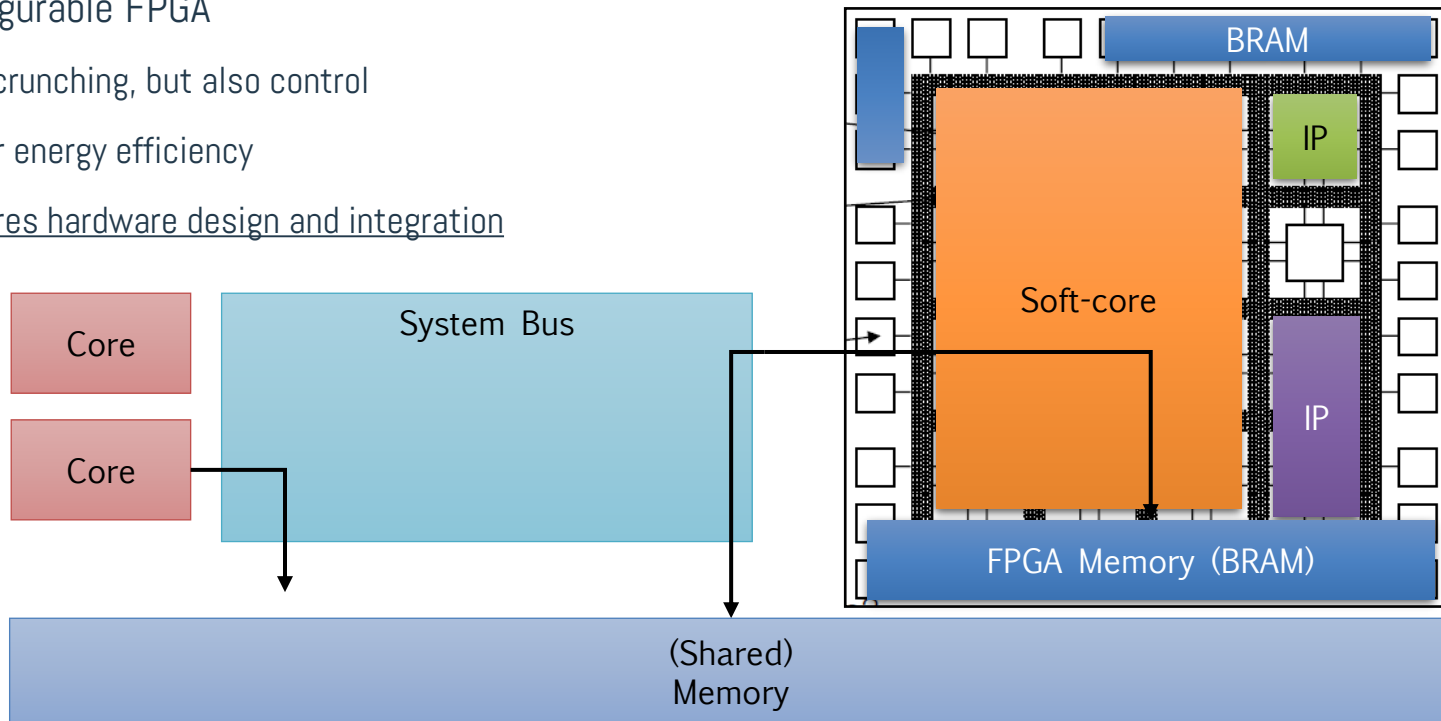
Xilinx UC+ / Kria

6 core ARM host

- › For control-based workload

Reconfigurable FPGA

- › Data crunching, but also control
- › Higher energy efficiency
- › Requires hardware design and integration





Our F1/10 cars



With GPGPUs

- › Bud => NVIDIA Orin
- › Kitt => NVIDIA Xavier NX



With FPGA

- › Thundershot F1/10 => Xilinx Kria



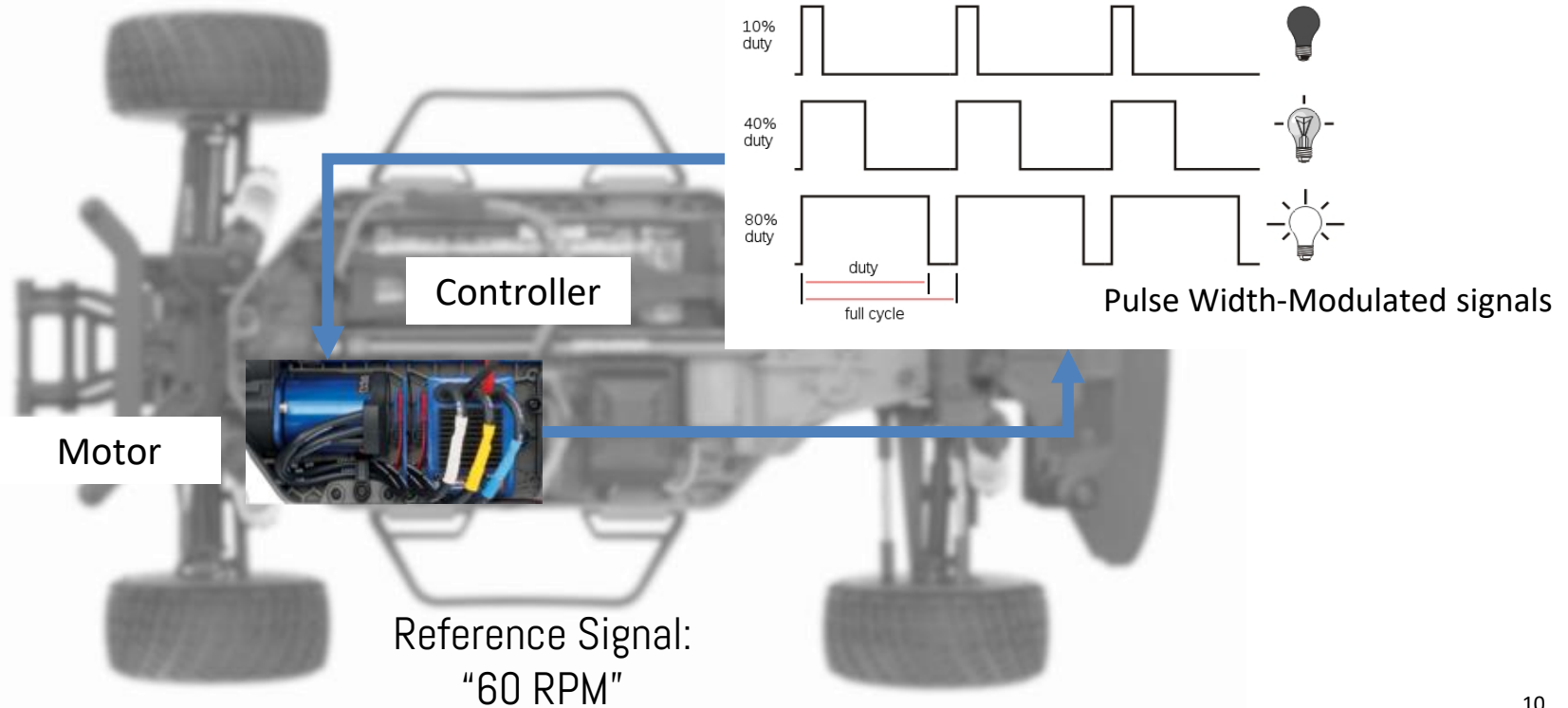
- › **Frankenstein/Frankie**
 - Whatever we want to test
 - Currently, cybersecurity





Actuation circuit

Remember: every vehicle is somehow unique!





PWM - D/A conversion (recap..?)

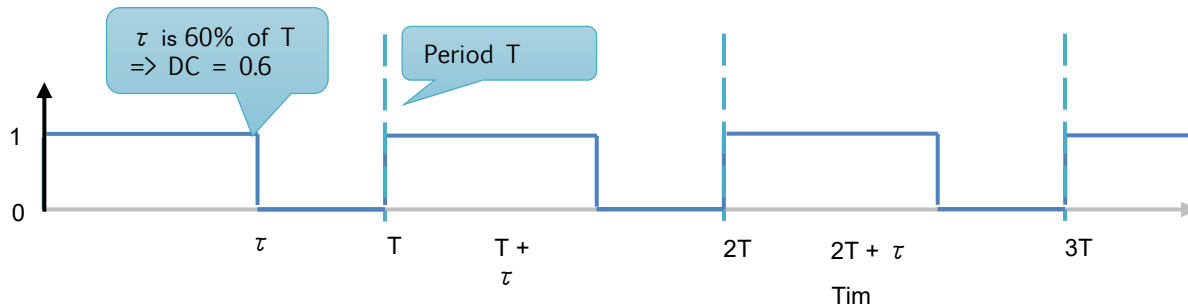
Generate a tension corresponding to a digital value stored in a register

- › Not easy! Use Pulse-Width Modulation (PWM)
 - (Almost) fully implementable in SW!

How it works

$$DC = \tau / T$$

1. Generate a periodic signal of amplitude 1 whose **duty cycle** is proportional to the digital value we want to convert
2. Give it to a low-pass filter, to average (such as Resistor-Capacitor - RC circuit)
3. ..and enjoy your analog signal! ☺

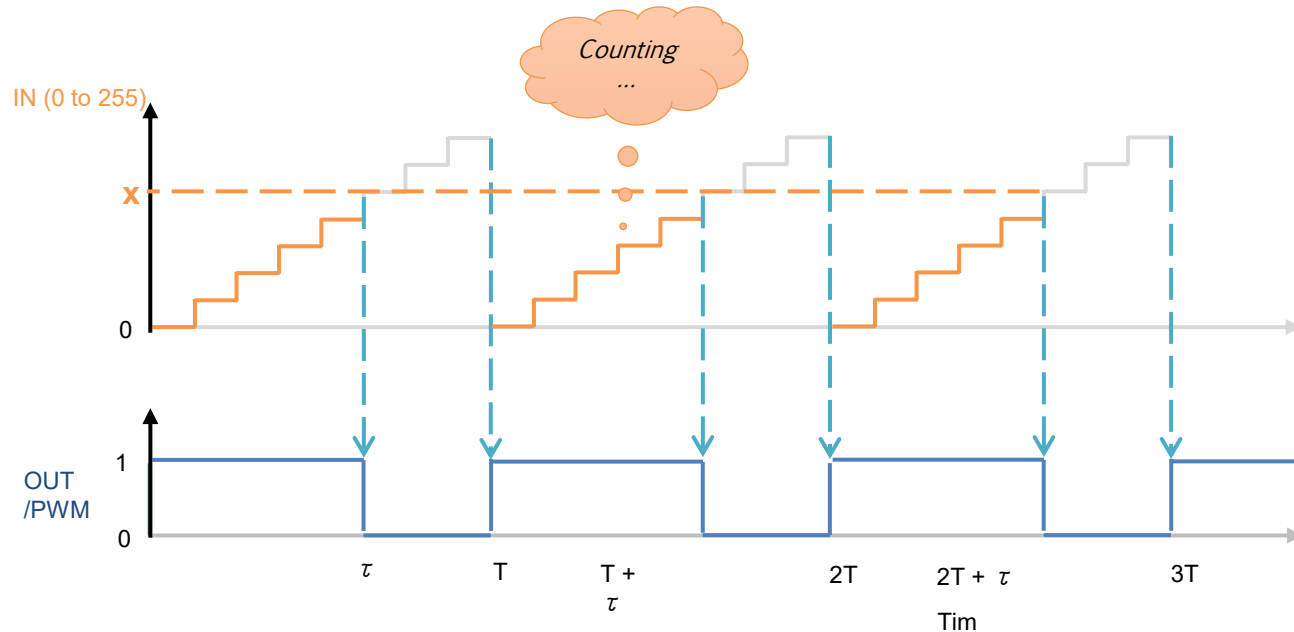




PWM (recap..?) – step 1

We need

- > A register that counts from 0 to 255 (8-bit)
- > An output port (bit) set to '1' and becoming '0' when input value matches the one of the register





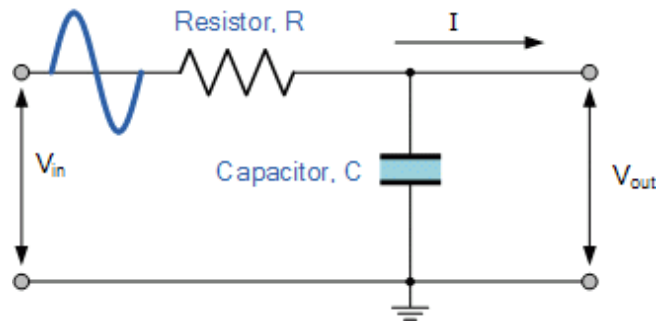
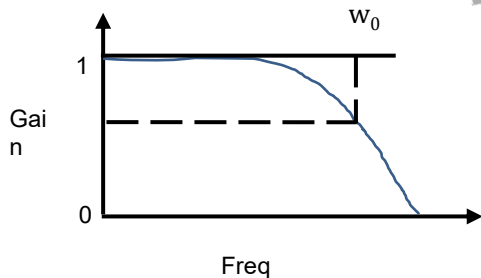
RC low-pass filter (recap..?)

Simple electric circuit with a Capacitor and a Resistance

> "Averages" the IN value

$$|V_{out}| = |V_{in}| \times \frac{1}{\sqrt{1 + \omega^2 R^2 C^2}}$$

Frequency domain



$$\text{Cutoff freq } \omega_0 = \frac{1}{RC}$$

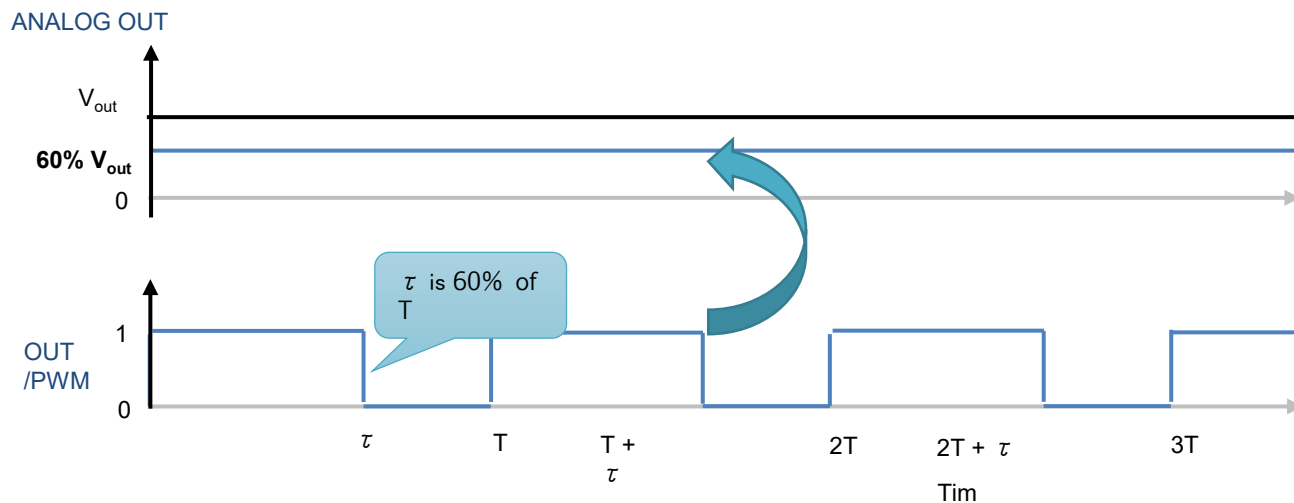


PWM (recap..?) – step 2

Now, compute the average for every T

- › Using a low-pass filter
- › Plug it to output port
- › *Et voilà*

Extremely useful in engine controls





Electronic Speed Controller – (V)ESC

Manages the brushless engines of the car

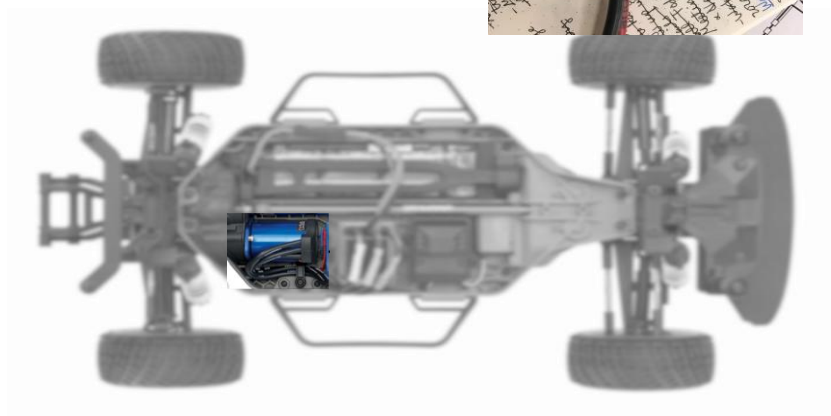
Programmed via a tool

- › Once-for-all

Accepts commands for lat/lng control

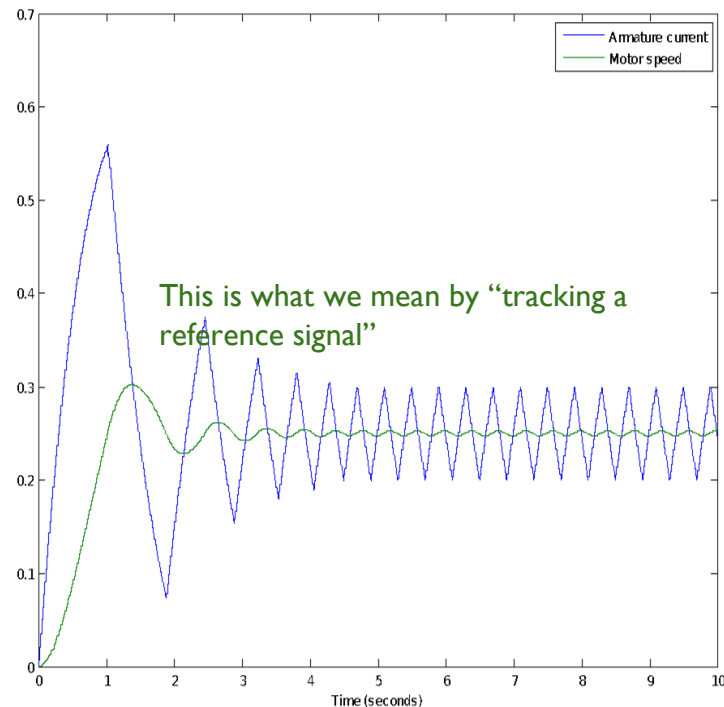
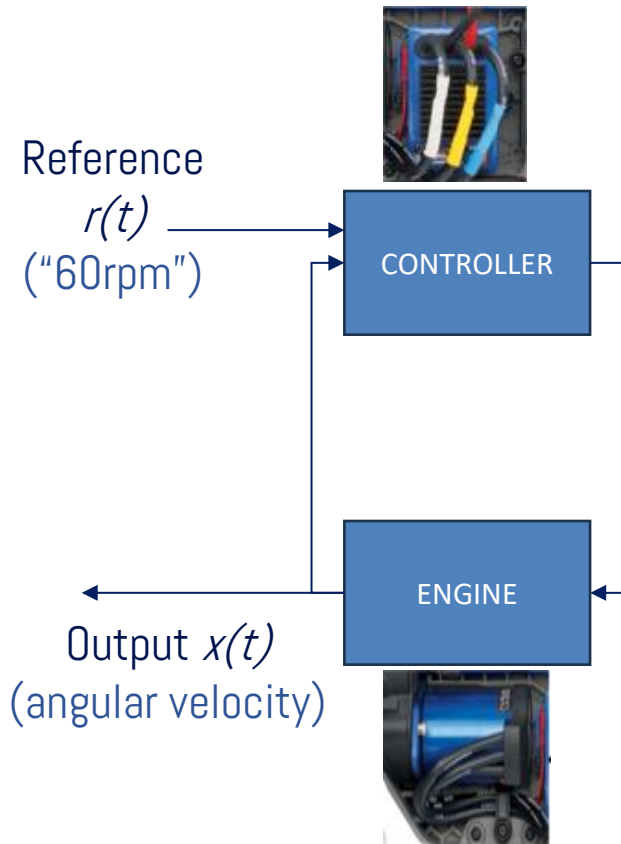
- › Acceleration & steering

<https://vesc-project.com/>





Actuation system W/VESC

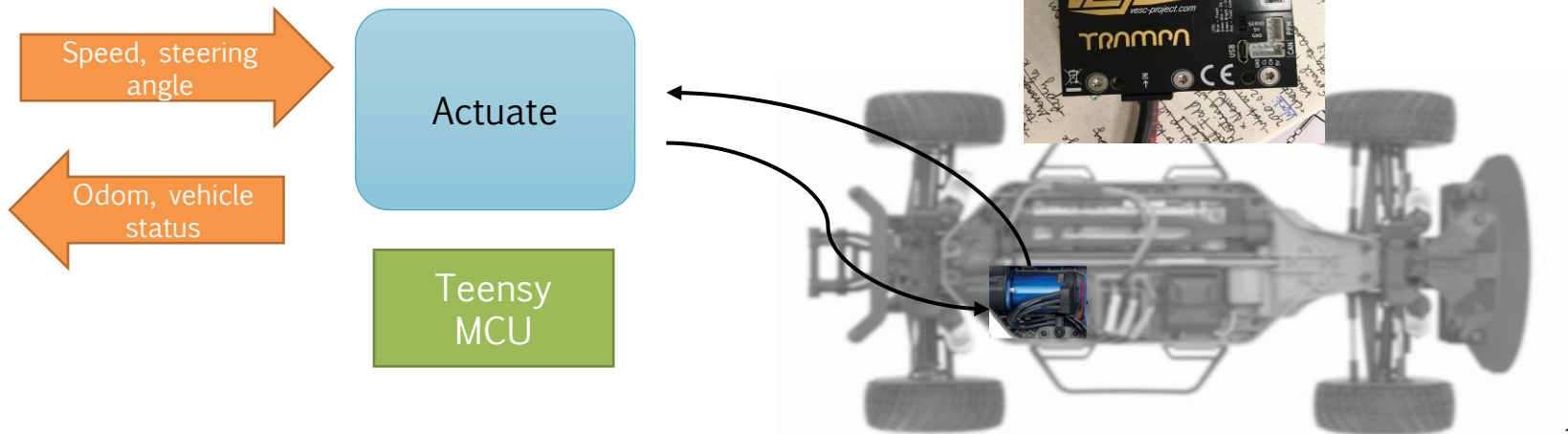




«Old» actuation based on Teensy μ controller

VESC accessed via UART

- › The navigation ECU had only CAN/serial port
- › Teensy converted the signal – teensy has no OS
- › Typical scenario, also real cars have legacy actuator ECUs!

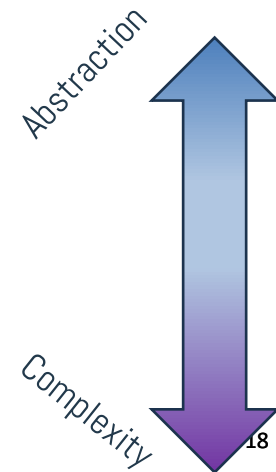
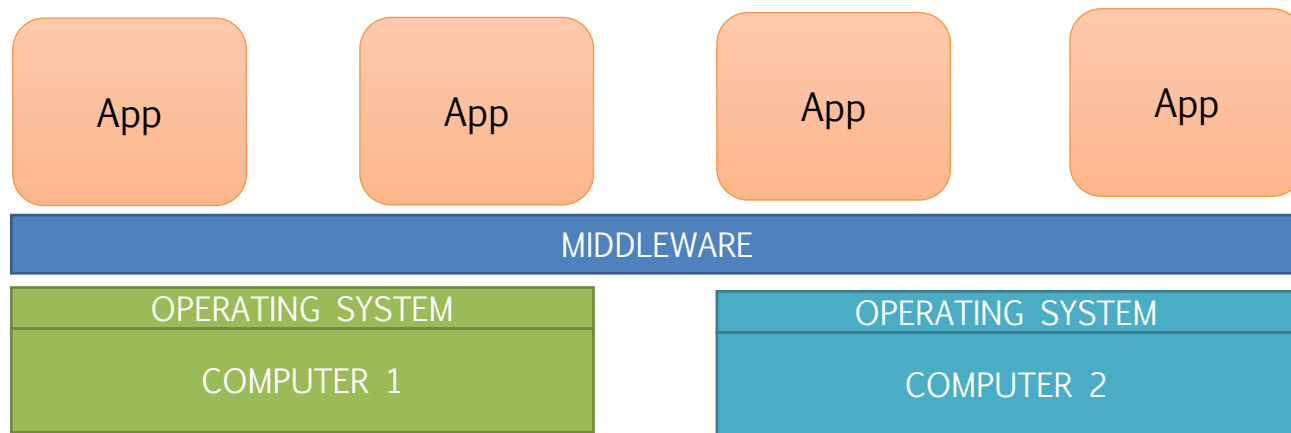




A quick travel...in programmer's mind

Computer science is all about hiding hardware

- › We typically represent it at the bottom of figures
- › At the top you have applications
- › In the middle, other components each of which has a specific responsibility

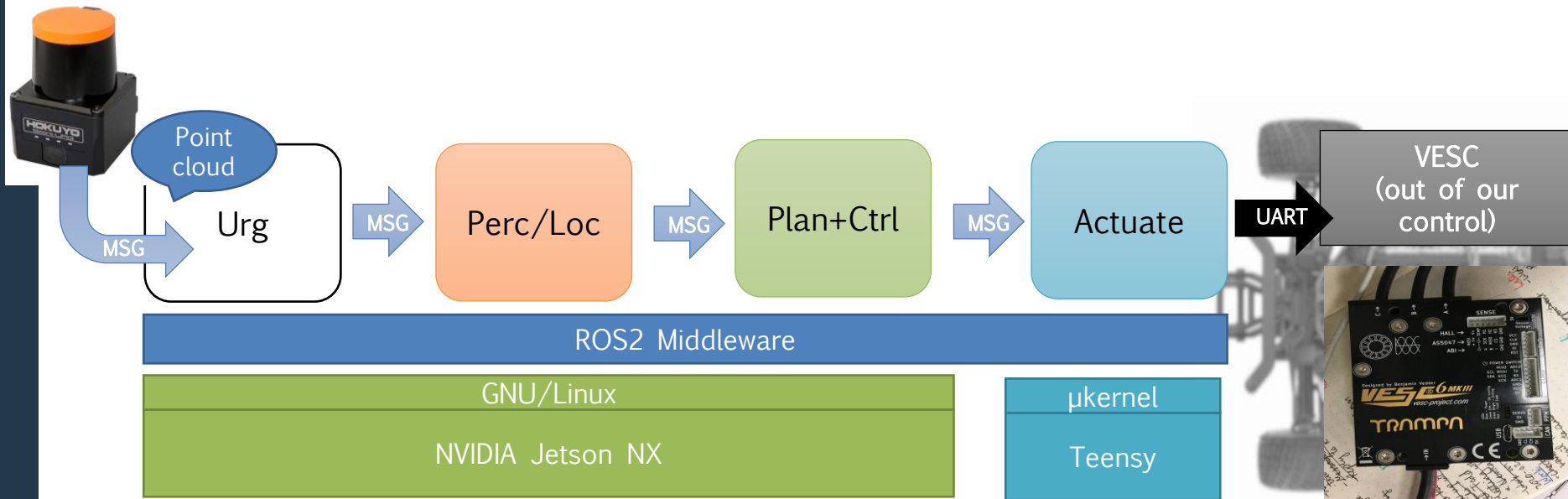




As applied to our AD stack

Robot Operating System (ROS)

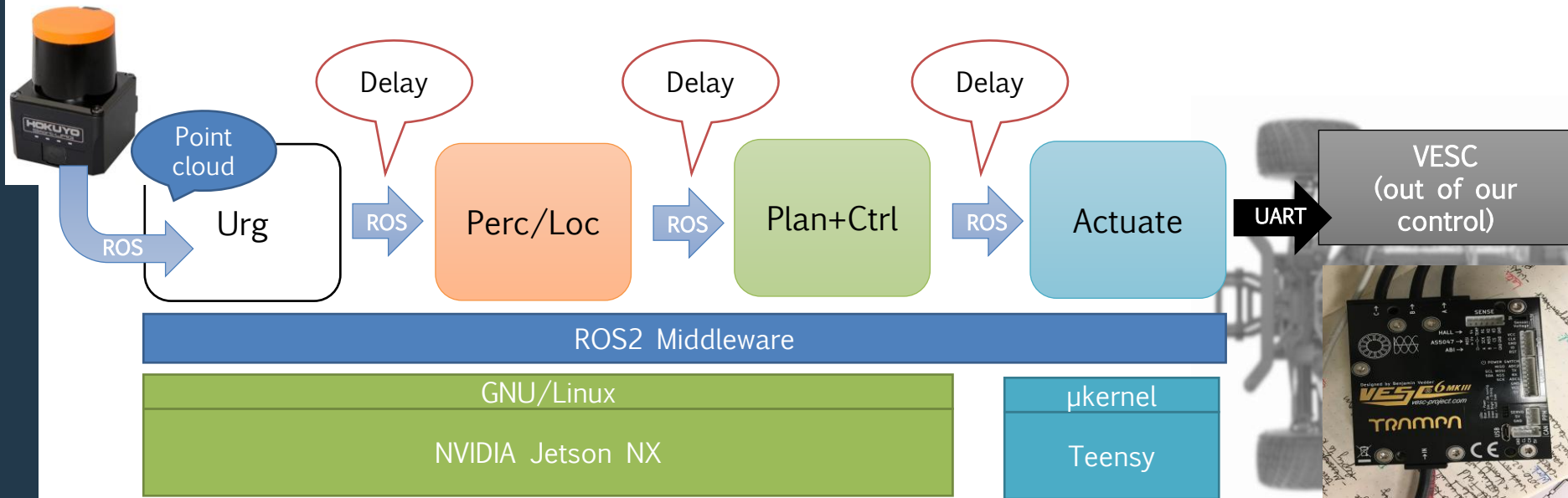
- › A middleware that hides the actual computer and OS we run on
- › Application components are (wrapped) called ROS **Nodes** that exchange ROS **Messages**
- › Most of sensors already publish ROS messages!





Actuation timings

- › LiDAR publishes @40Hz (25ms)
- › Teensy loop @?? Hz
 - We don't care! If we don't send a signal, it holds the previous one
- › Our pipeline must meet 25ms deadline...incl. ROS delays!

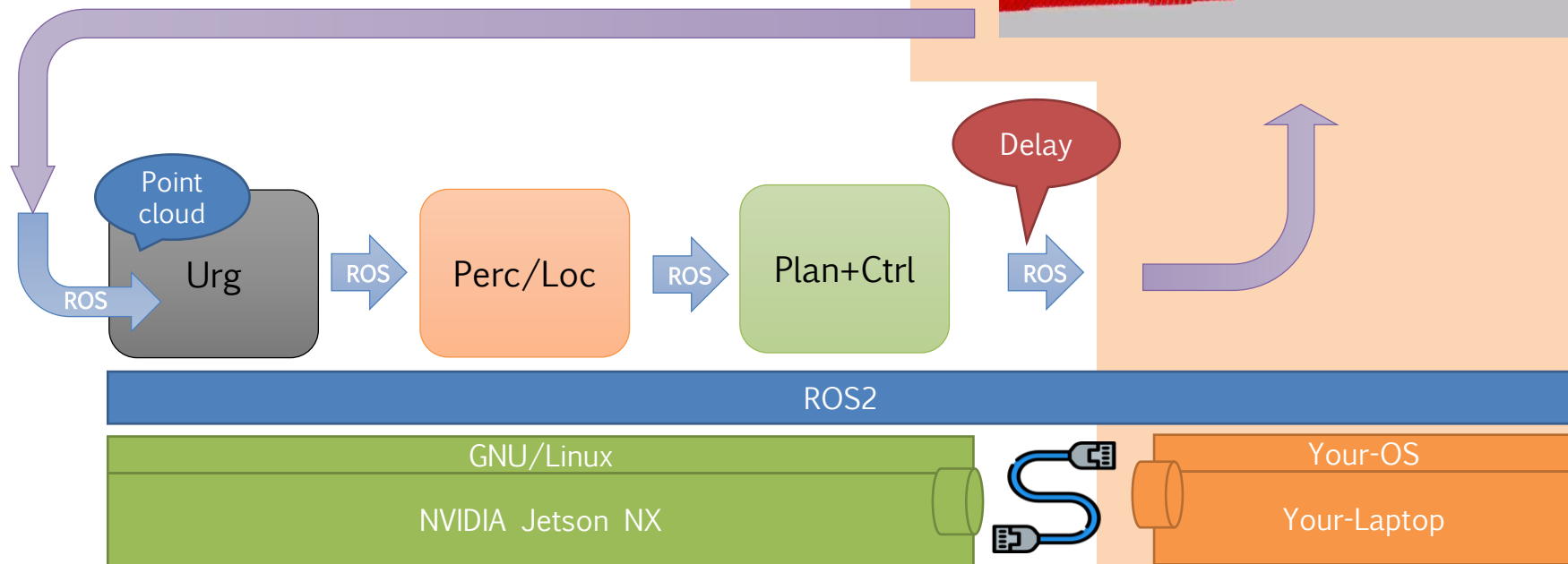
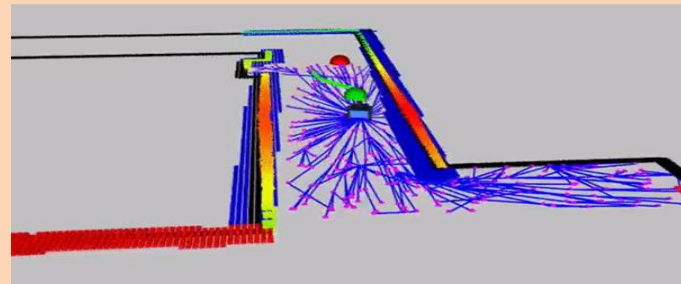




Interaction with a simulator

Most of simulators have can exchange ROS messages!

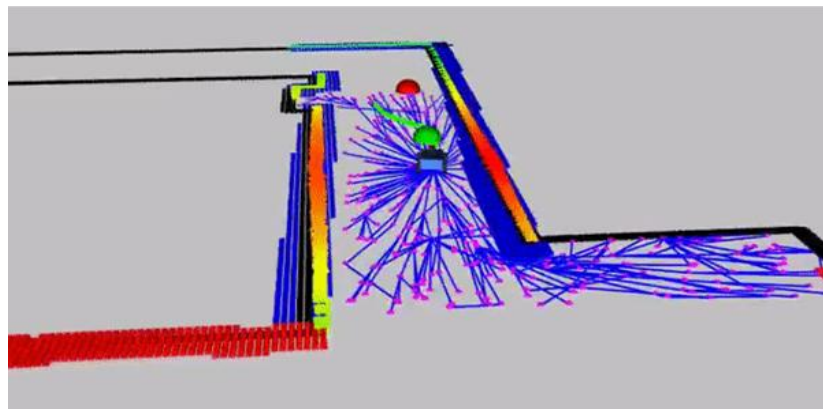
- › Can receive commands
- › Send data from *simulated* sensors
- › ROS ensures that every exchange is machine and OS independant





F1/10 Gym

- › Lightweight 2D simulator built in Python
- › Faster than real-time execution (30x realtime)
- › Runs multiple vehicle instances
- › Publishes laser scan and odometry data
- › Built for fast prototyping





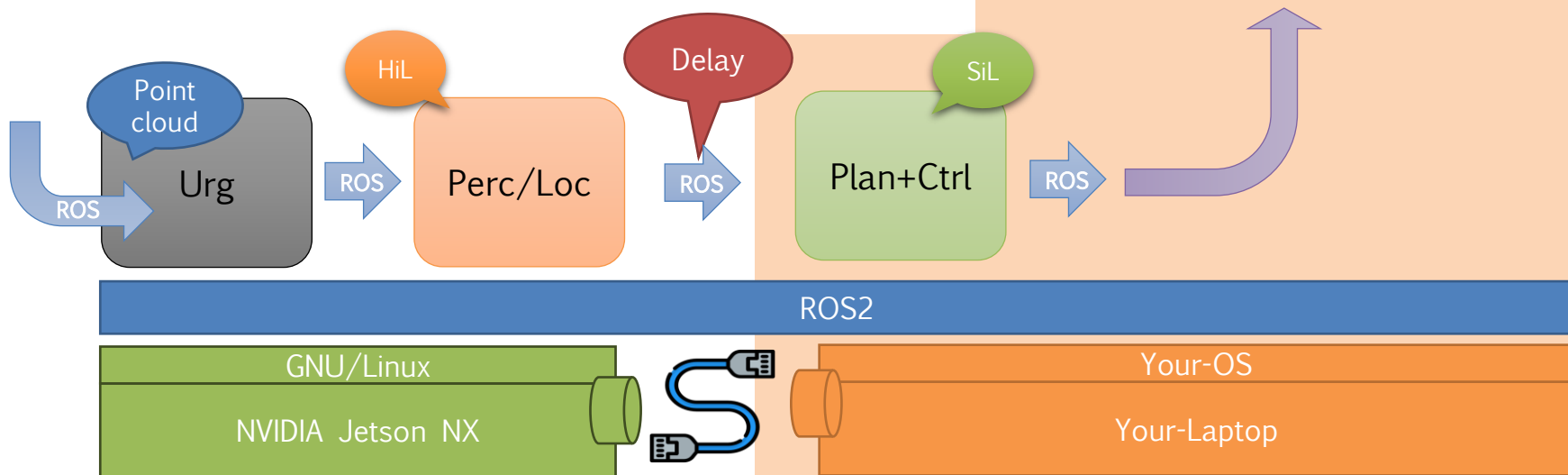
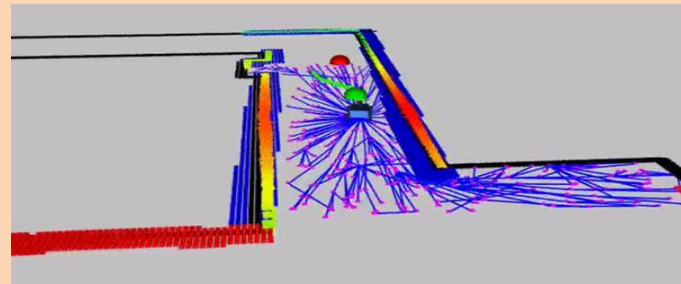
Where to run application components?

Hardware-in-the-loop - HiL

- › Run them in the very same ECU hardware
- › With delays due to comm, but execution is the same than on real ECU

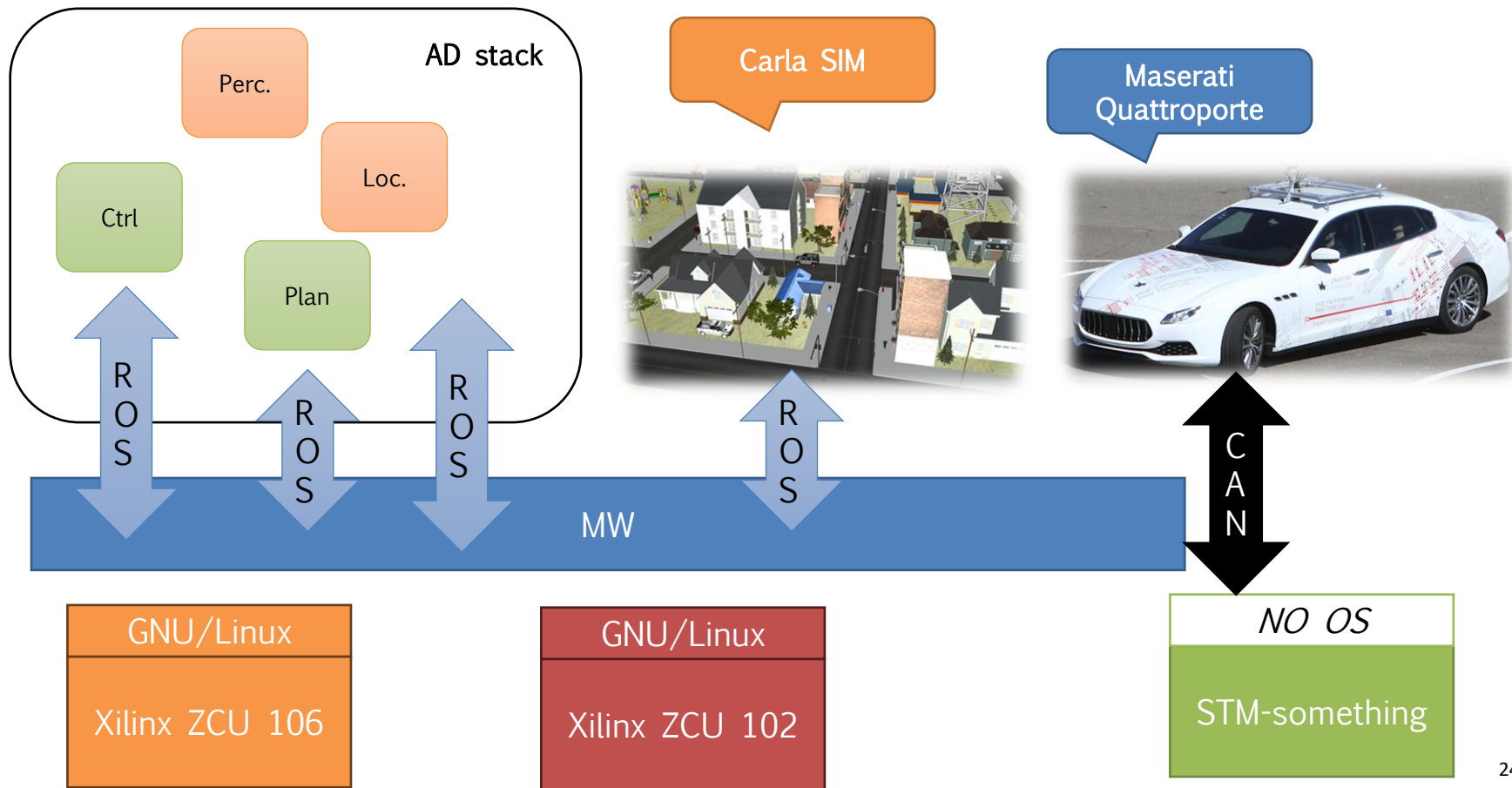
Software-in-the-loop - SiL

- › Run them in the same computer where the simulator resides



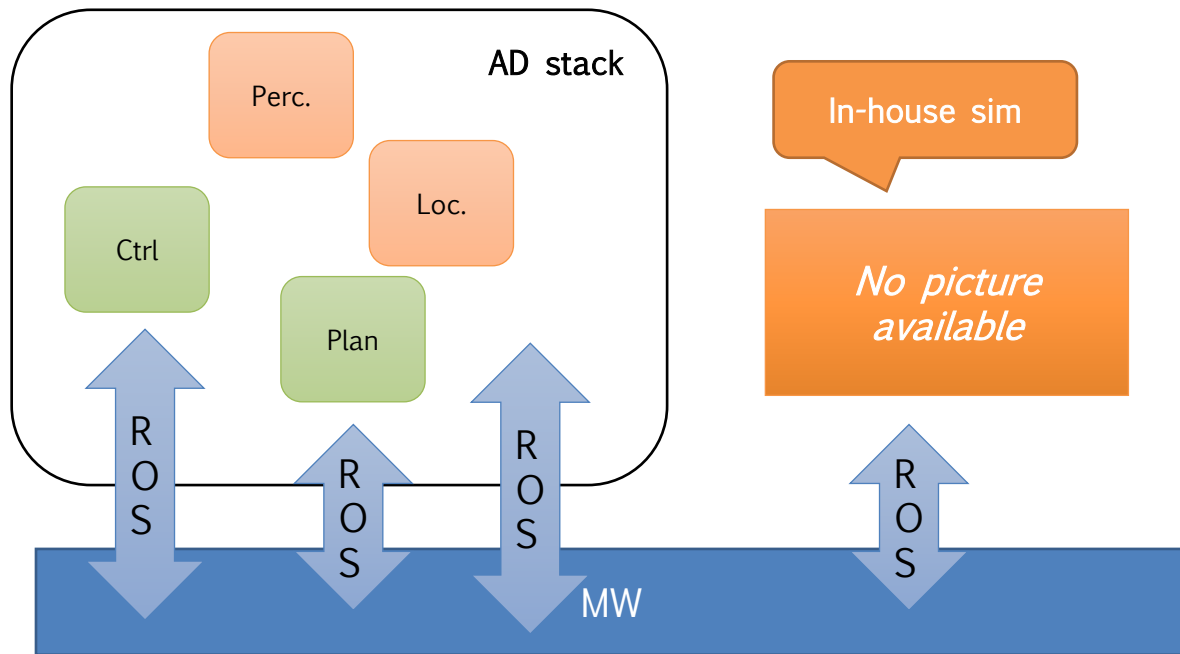


The Thundershot project – A more complex example





Formula Student



In-house sim

No picture available

MR24-DL



G
P
I
O

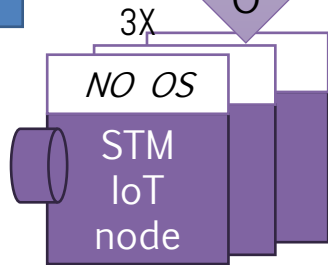
?
?



Ethernet



CAN





Assignments

I can send you a quiz which you need to solve and send back to me

- › By friday

..Or...you can challenge yourself in one of the following (with support by me and the guys from the lab)

- › Building a simple node for AD components (e.g., pure pursuit, follow-the-gap) – Python or C++
- › Implement some model of a vehicle (Matlab/Simulink)
- › Your ideas...?

No deadline for the latter, the earlier the better



References



Course website

- › <https://github.com/HiPeRT/F1tenth-PhD>

My contacts

- › paolo.burgio@unimore.it
- › <http://hipert.mat.unimore.it/people/paolob/>

Resources

- › <https://f1tenth.org>
- › A "small blog"
 - <http://www.google.com>